

Билет 17. Когерентность кэшей. Протокол MESI. Cache ping-pong. False sharing

0. Формулировка билета

Когерентность кэшей. Протокол MESI. Cache ping-pong. Проблема false sharing и её решение.

Главная идея:

У каждого ядра есть свои кэши.

Одна и та же область памяти может иметь копии в кэшах разных ядер.

Когерентность кэшей нужна, чтобы сохранять иллюзию единой общей памяти.

Но важно различать две вещи:

корректность:

процессор поддерживает когерентность,

поэтому мы можем думать, что одна переменная – это одна переменная

производительность:

за поддержание когерентности приходится платить коммуникацией между ядрами

Этот билет в первую очередь про производительность многопоточного кода.

1. Зачем нужны кэши

Процессорное ядро очень быстрое, а оперативная память намного медленнее.

Если каждый раз ходить в RAM, процессор будет простаивать.

Поэтому между ядром и памятью есть кэши:

CPU core

↓

L1 cache

↓

L2 cache

↓

L3 cache

↓

RAM

Кэш — это маленькое быстрое хранилище рядом с ядром.

Если нужные данные уже есть в кэше:

```
cache hit
```

операция быстрая.

Если данных нет:

```
cache miss
```

нужно идти ниже по иерархии, это дороже.

2. Локальность

Кэши работают хорошо из-за локальности.

Есть два основных вида локальности.

Temporal locality

Если мы недавно обращались к данным, вероятно, скоро обратимся снова.

Пример:

```
for (int i = 0; i < 1000; ++i) {  
    sum += x;  
}
```

Переменная `x` будет многократно использоваться.

Spatial locality

Если мы обратились к адресу, вероятно, скоро обратимся к соседним адресам.

Пример:

```
for (int i = 0; i < n; ++i) {  
    sum += a[i];  
}
```

Массив читается подряд.

Поэтому процессор загружает в кэш не один байт и не один `int`, а сразу блок соседних данных.

3. Cache line

Кэш работает не отдельными переменными, а **кэш-линиями**.

Cache line — минимальная единица обмена между памятью и кэшем.

Обычно размер:

64 bytes

То есть если мы читаем один `int`, процессор может принести в кэш всю линию:

адреса $X \dots X + 63$

Следствие:

Когерентность тоже работает на уровне cache line, а не отдельной переменной.

Это очень важно.

Если две разные переменные лежат в одной cache line, для процессора они частично “связаны” с точки зрения кэш-протокола.

4. Проблема нескольких кэшей

На одном ядре всё просто:

core #1
cache
memory

Но на многопроцессорной системе:

core #1	core #2	core #3	core #4
L1	L1	L1	L1
\			/
	memory		

Одна и та же cache line может быть скопирована в кэшах разных ядер.

Например, переменная:

```
int x = 0;
```

лежит в cache line **L**.

Ядро 1 прочитало **x**:

```
core #1 cache: L with x = 0
```

Ядро 2 прочитало **x**:

```
core #2 cache: L with x = 0
```

Теперь обе копии одинаковые.

Но что если ядро 1 записало:

```
x = 1;
```

Тогда копия в core #2 устарела.

Нужно не допустить ситуации:

```
core #1 думает: x = 1  
core #2 думает: x = 0
```

как будто это навсегда две разные реальности.

Для этого нужен протокол когерентности.

5. Когерентность кэшей

Cache coherence — это механизм, который поддерживает согласованность копий одной cache line в разных кэшах.

Он должен сохранять иллюзию:

Есть одна общая память, а не независимые копии у каждого ядра.

Упрощённо, у когерентности есть два инварианта:

1. Если у линии есть несколько копий в кэшах,
то эти копии одинаковые и совпадают с памятью.
2. Если какая-то копия линии отличается от памяти,
то она единственная.

Первый случай:

несколько читателей
все видят одинаковую чистую копию

Второй случай:

один писатель
у него единственная грязная копия

Это похоже на readers-writer lock на уровне cache line:

или много читателей
или один писатель

6. Write-invalidate

Распространённая идея протоколов когерентности — **write-invalidate**.

Если ядро хочет записать в cache line, оно должно получить её в эксклюзивное владение.

Для этого оно инвалидирует копии этой линии в других кэшах.

Схема:

```
core #1 хочет писать в L

core #1:
    получает L эксклюзивно

core #2, core #3:
    их копии L становятся invalid
```

После этого core #1 может писать.

Если core #2 потом снова захочет прочитать эту линию, ему придётся заново получить актуальную копию.

7. MSI

Простейшая модель протокола — MSI.

У каждой cache line в кэше есть состояние:

M – Modified
S – Shared
I – Invalid

Modified

M = Modified

Линия изменена в этом кэше.

Свойства:

эта копия отличается от памяти
эта копия единственная
другие кэши не имеют валидной копии

То есть это состояние “я единственный владелец и у меня грязная копия”.

Shared

S = Shared

Линия чистая.

Свойства:

копия совпадает с памятью
такая копия может быть в нескольких кэшах

Это состояние “можно читать многим”.

Invalid

I = Invalid

Копия недействительна.

Свойства:

эту cache line нельзя использовать
при чтении/записи нужно заново получать линию

8. MESI

MESI добавляет к MSI ещё одно состояние:

E – Exclusive

Полный набор:

M – Modified
E – Exclusive
S – Shared
I – Invalid

Exclusive

E = Exclusive

Линия чистая, совпадает с памятью, но находится только в этом кэше.

Свойства:

копия не грязная
она единственная
других копий нет

Зачем нужно состояние E ?

Чтобы ускорить сценарий:

ядро прочитало линию
потом это же ядро захотело записать в неё

Если линия в состоянии **S**, для записи нужно послать invalidation другим кэшам.

Но если линия в состоянии **E**, других копий нет, значит можно перейти в **M** локально:

$E \rightarrow M$

без лишней коммуникации.

То есть **E** — оптимизация для случая “линия пока только у меня”.

9. Базовые переходы MESI

Не нужно заучивать весь автомат, но нужно понимать основные сценарии.

Чтение линии, которой нет в кэше

$I + \text{read}$

Если ни у кого другой копии нет:

$I \rightarrow E$

Если копии есть у других:

$I \rightarrow S$

Запись в линию, которой нет в кэше

$I + \text{write}$

Ядро должно получить линию эксклюзивно:

$I \rightarrow M$

Другие копии инвалидируются.

Запись в shared line

S + write

Если линия shared, значит она может быть у других кэшей.

Для записи нужно инвалидировать другие копии:

S → M

У других:

S → I

Запись в exclusive line

E + write

Если линия exclusive, других копий нет.

Можно перейти:

E → M

без обращения к другим ядрам.

Чтение modified line другим ядром

Если core #1 держит линию в **M**, а core #2 хочет её прочитать, то память может быть устаревшей.

Поэтому core #1 должен отдать актуальные данные.

Упрощённо:

core #1: M → S
core #2: I → S

А данные либо записываются обратно в память, либо передаются между кэшами в зависимости от реализации.

10. Почему это важно для синхронизации

С точки зрения корректности мы можем думать:

есть одна переменная `x`

Но с точки зрения производительности:

`x` — это `cache line`,
которая может перемещаться между кэшами ядер

Если несколько ядер постоянно пишут в одну линию, протокол когерентности вынужден постоянно передавать владение.

Это дорого.

Поэтому:

Оптимизировать многопоточный код — значит уменьшать ненужную коммуникацию между кэшами.

11. Cache ping-pong

Cache ping-pong — ситуация, когда одна `cache line` постоянно “прыгает” между ядрами.

Пример:

```
std::atomic<int> counter{0};

void worker() {
    for (int i = 0; i < 1000000; ++i) {
        counter.fetch_add(1);
    }
}
```

Если много потоков делают `fetch_add` одного счётчика, каждый поток хочет писать в одну и ту же `cache line`.

Сценарий:

core #1 делает `fetch_add`:
получает линию `counter` в М
копии других ядер инвалидируются

```
core #2 делает fetch_add:  
    теперь core #2 получает эту линию в M  
    копия core #1 инвалидируется
```

```
core #3 делает fetch_add:  
    линия снова переезжает к core #3
```

Линия постоянно перемещается:

```
core #1 → core #2 → core #3 → core #1 → ...
```

Это и есть cache ping-pong.

Результат:

```
много коммуникации  
много инвалидаций  
плохое масштабирование
```

12. CAS как запись

Важно понимать стоимость CAS.

`compare_exchange` кажется условной операцией:

```
если значение равно expected:  
    записать desired  
иначе:  
    не записывать
```

Но на уровне кэш-когерентности CAS обычно нужно считать записью.

Почему?

Потому что ядру заранее нужно получить cache line в эксклюзивное владение.

Оно ещё не знает, успешным будет CAS или нет.

Поэтому даже неуспешный CAS может:

забрать линию в exclusive/modified
инвалидировать копии других ядер
создать cache ping-pong

Следствие:

Если в spinlock все потоки крутятся на `exchange` или CAS, они постоянно пишут в одну линию и создают сильный cache ping-pong.

13. Spinlock и cache ping-pong

Плохой spinlock:

```
class SpinLock {
public:
    void lock() {
        while (locked.exchange(true)) {
        }
    }

    void unlock() {
        locked.store(false);
    }

private:
    std::atomic<bool> locked{false};
};
```

Почему плохо?

Пока lock занят, все ожидающие потоки делают:

```
exchange(true)
```

Это RMW-операция.

Даже если lock уже `true`, каждый `exchange` — это запись.

Значит, все ожидающие ядра постоянно борются за одну cache line `locked`.

Итог:

cache ping-pong
много инвалидаций
плохое масштабирование

14. TTAS: test-test-and-set

Улучшение — **TTAS**: test-test-and-set.

Идея:

пока lock занят:
крутимся на обычном load

когда кажется, что lock свободен:
пробуем exchange

Код:

```
class SpinLock {
public:
    void lock() {
        while (true) {
            while (locked.load(std::memory_order_relaxed)) {
                // просто читаем
            }

            if (!locked.exchange(true, std::memory_order_acquire)) {
                return;
            }
        }
    }

    void unlock() {
        locked.store(false, std::memory_order_release);
    }

private:
    std::atomic<bool> locked{false};
};
```

Почему лучше?

Пока lock занят, все читают одну shared line:

```
S state
```

Чтения могут обслуживаться из локальных кэшей.

Они не требуют постоянного эксклюзивного владения.

Писать пытаются только когда увидели:

```
locked == false
```

Это уменьшает cache ping-pong.

Но проблема полностью не исчезает.

Когда владелец делает:

```
locked.store(false)
```

все ожидающие видят освобождение и почти одновременно пробуют `exchange`.

Это похоже на:

```
thundering herd
```

Много потоков просыпаются и борются за одну линию.

15. True sharing

True sharing — настоящее разделение.

Это когда несколько потоков действительно работают с одной и той же переменной или структурой.

Примеры:

```
std::atomic<int> counter;  
std::mutex m;  
std::atomic<bool> locked;
```

Здесь cache ping-pong ожидаем:

все реально обращаются к одному shared state

Полностью убрать коммуникацию нельзя, потому что данные действительно общие.

Можно только уменьшить частоту:

локальные счётчики вместо одного глобального
batching
sharding
очереди на worker
более масштабируемые lock-и

16. False sharing

False sharing — ложное разделение.

Это ситуация, когда разные потоки работают с разными переменными, но эти переменные лежат в одной cache line.

Логически данные независимы.

Но процессор видит:

одна cache line

Пример:

```
struct Counters {  
    std::atomic<long long> a;  
    std::atomic<long long> b;  
};  
  
Counters counters;
```

Поток 1 пишет:

```
counters.a.fetch_add(1);
```

Поток 2 пишет:

```
counters.b.fetch_add(1);
```

Логически:

а и b независимы

Но если **a** и **b** попали в одну cache line, то запись в **a** инвалидирует линию у потока, который работает с **b**.

И наоборот.

Получается cache ping-pong без реального разделения данных.

17. Пример false sharing

Пусть cache line 64 байта.

```
struct Counter {  
    std::atomic<long long> value;  
};  
  
Counter counters[2];
```

counters[0] и **counters[1]** могут лежать рядом в памяти, в одной cache line.

Поток 1:

```
for (...) {  
    counters[0].value.fetch_add(1);  
}
```

Поток 2:

```
for (...) {  
    counters[1].value.fetch_add(1);  
}
```

Каждый поток обновляет свой счётчик.

Но из-за одной cache line процессор вынужден инвалидировать всю линию.

Схема:

```
core #1 пишет counters[0]  
линия переходит к core #1
```



```
core #2 пишет counters[1]
    та же линия переходит к core #2

core #1 снова пишет counters[0]
    линия снова переходит к core #1
```

Хотя переменные разные.

18. Решение false sharing

Основное решение:

Разнести независимые горячие переменные по разным cache line.

Например, через выравнивание:

```
struct alignas(64) PaddedCounter {
    std::atomic<long long> value;
};

PaddedCounter counters[2];
```

Теперь каждый счётчик занимает отдельную cache line.

Более переносимый вариант:

```
#include <new>

struct alignas(std::hardware_destructive_interference_size) PaddedCounter {
    std::atomic<long long> value;
};
```

`std::hardware_destructive_interference_size` — это стандартная константа, которая показывает размер, полезный для разведения объектов, чтобы они не мешали друг другу в кэше.

Её смысл:

разнести горячие переменные,
между которыми возможен destructive interference

То есть false sharing лечится:

```
padding  
alignment  
разнесением горячих полей
```

19. Осторожно: padding не бесплатный

Выравнивание помогает, но у него есть цена.

Если мы делаем:

```
struct alignas(64) PaddedCounter {  
    std::atomic<long long> value;  
};
```

то один маленький счётчик может занимать 64 байта.

Минусы:

```
больше памяти  
хуже помещаемость в кэш  
больше pressure на cache/TLB
```

Поэтому padding нужен не везде, а только для горячих полей, которые действительно часто пишутся разными потоками.

20. Пример: per-thread counters

Вместо одного глобального счётчика:

```
std::atomic<long long> global_counter;
```

можно сделать счётчики по потокам:

```
struct alignas(std::hardware_destructive_interference_size) Counter {  
    std::atomic<long long> value{0};  
};  
  
std::vector<Counter> counters(num_threads);
```

Каждый поток пишет только в свой счётчик.

```
counters[thread_id].value.fetch_add(1, std::memory_order_relaxed);
```

А итог считаем редко:

```
long long total = 0;

for (auto& c : counters) {
    total += c.value.load(std::memory_order_relaxed);
}
```

Плюс:

меньше contention
меньше cache ping-pong
лучшее масштабирование

Минус:

получение точной суммы дороже
значение может быть не мгновенно централизованным

21. False sharing в структурах данных

False sharing может появиться не только в счётчиках.

Примеры:

```
поля в thread pool:
    stop flag
    queue size
    статистика worker-ов

lock-free queue:
    head и tail рядом в одной cache line

ticket lock:
    owner_ticket и next_ticket рядом

массив per-worker структур:
    workers[i].stats
    workers[i + 1].stats
```

Если разные ядра часто пишут в разные поля одной линии, получаем false sharing.

Решение:

```
разнести горячие поля
добавить padding
использовать alignas
разделить read-mostly и write-heavy данные
```

22. Cache locality и правильная сторона оптимизации

Важно не делать вывод:

```
надо всегда всё разносить по разным cache line
```

Нет.

Иногда наоборот выгодно держать данные рядом.

Если один поток часто читает несколько полей вместе:

```
struct Point {
    double x;
    double y;
};
```

то хорошо, что `x` и `y` лежат рядом.

Это spatial locality.

Проблема возникает, когда:

```
разные ядра часто пишут в разные поля одной линии
```

То есть критерий такой:

```
один поток часто использует вместе → держать рядом
разные потоки часто пишут независимо → разносить
```

23. Когерентность vs memory model

Когерентность кэшей и модель памяти — разные уровни.

Когерентность кэшей отвечает:

как процессор поддерживает согласованность копий одной cache line

Модель памяти C++ отвечает:

какие обращения к памяти видны каким потокам
какие переупорядочивания допустимы
что значит data race
что создаёт happens-before

Когерентность не отменяет memory ordering.

Даже если кэши когерентны, это не значит, что любой многопоточный код корректен.

Например:

```
data = 42;  
ready = true;
```

Другой поток может видеть `ready`, но не иметь корректной синхронизации для `data`, если нет proper memory order.

Поэтому:

cache coherence помогает сохранять иллюзию единой памяти на уровне железа
memory model задаёт правила корректной синхронизации на уровне C++

24. Что сказать коротко

Кэши нужны, потому что память намного медленнее процессорного ядра. Кэш работает не отдельными переменными, а кэш-линиями, обычно около 64 байт. На многоядерной системе одна и та же cache line может иметь копии в кэшах разных ядер, поэтому нужен протокол когерентности, который сохраняет иллюзию единой общей памяти.

Когерентность поддерживает инвариант: либо линия может быть в нескольких кэшах в чистом shared-состоянии, либо есть единственная грязная копия у одного ядра. Это похоже на readers-writer lock на уровне cache line. Протокол MESI использует состояния Modified, Exclusive, Shared, Invalid. **M** — грязная единственная копия, **E** — чистая единственная копия, **S** — чистая копия, которая может быть у нескольких ядер, **I** — невалидная линия.

При записи ядро должно получить cache line эксклюзивно и инвалидировать копии у других ядер. Если несколько ядер постоянно пишут в одну линию, линия начинает “прыгать” между кэшами — это cache ping-pong. Он возникает на горячих общих атомиках, mutex/spinlock, глобальных

счётчиках. CAS и `exchange` нужно считать записью, даже если CAS неуспешный, потому что ядро всё равно должно получить линию в эксклюзивное владение.

False sharing — это cache ping-pong без настоящего логического разделения: разные потоки пишут в разные переменные, но переменные лежат в одной cache line. Решение — разнести горячие независимые поля по разным cache line через padding/ `alignas`, например `alignas(std::hardware_destructive_interference_size)`. Но padding не бесплатный: он увеличивает расход памяти и может ухудшить locality, поэтому его применяют к горячим полям, которые часто пишутся разными потоками.

25. Мини-проверка

1. Зачем процессору нужны кэши?
2. Что такое cache line?
3. Почему когерентность работает на уровне cache line, а не отдельной переменной?
4. Что такое cache coherence?
5. Какие два инварианта когерентности полезно помнить?
6. Что означают состояния `M`, `E`, `S`, `I` в MESI?
7. Зачем в MESI нужно состояние `E`?
8. Что происходит, когда ядро хочет записать в shared cache line?
9. Что такое cache ping-pong?
10. Почему CAS и `exchange` нужно считать записью с точки зрения кэшей?
11. Что такое false sharing?
12. Как решать false sharing?